

# รายงานการปฏิบัติการ

วิชา: CP353002 Principles of Software Design

ผู้จัดทำ: นางสาว อาทิตยา โคตรธินสาร รหัสนักศึกษา 643021259-2 Section1

## ส่วนที่ 1: หลักการออกแบบ

### 1. การประยุกต์ใช้ SOLID Principles กับ Entity

การออกแบบระบบในครั้งนี้ได้นำหลักการออกแบบเชิงวัตถุ (SOLID Principles) มาประยุกต์ใช้เพื่อความยืดหยุ่นและการดูแลรักษาโค้ดในระยะยาว ดังนี้

- **S (Single Responsibility Principle)** : แต่ละคลาสหรือตารางควรมีหน้าที่รับผิดชอบเพียงอย่างเดียว เช่น การแยกข้อมูลรายละเอียดสินค้า (ProductDetail) และข้อมูลรีวิว (Review) ออกจากคลาสหลักอย่าง Product ทำให้ Entity ไม่แบกรับภาระหน้าที่ที่หลากหลายจนเกินไป
- **O (Open/Closed Principle)** : ระบบควรเปิดรับการขยายตัวแต่ปิดรับการแก้ไขโครงสร้างเดิม เช่น เมื่อต้องการเพิ่มตารางรีวิวสินค้า (Review) สามารถสร้าง Entity ขึ้นมาใหม่ได้ทันทีโดยไม่ต้องแก้ไขโค้ดโครงสร้างเดิมภายใน Product
- **L (Liskov Substitution Principle)** : การใช้งาน Repository Interfaces สามารถนำไปใช้แทนที่กันได้ได้อย่างถูกต้องในทุก ๆ Entity โดยไม่ทำให้การทำงานของโปรแกรมผิดเพี้ยน
- **I (Interface Segregation Principle)** : การแยก Interface ของ Repository ออกเป็นสัดส่วนเฉพาะตัว เช่น ProductRepository, ProductDetailRepository และ ReviewRepository แทนที่จะยุบรวมทุกอย่างไว้ใน Interface เดียว
- **D (Dependency Inversion Principle)** : คลาสในชั้น Service จะต้องพึ่งพา Abstraction (เช่น ProductRepository ที่เป็น Interface) แทนการผูกติดกับ Implementation หรือคลาสจริงโดยตรง

### 2. ความแตกต่างระหว่าง 1:1 กับ 1:N

- **ความสัมพันธ์แบบ One-to-One (1:1)** : เป็นความสัมพันธ์ที่ข้อมูลฝั่งหนึ่งเชื่อมโยงกับอีกฝั่งได้เพียง 1 รายการเท่านั้น เช่น Product (1) มีความสัมพันธ์กับ ProductDetail (1) ซึ่งใช้ในกรณีที่ข้อมูลเสริมของสินค้า (เช่น น้ำหนัก, ขนาด, การรับประกัน) ที่ไม่ได้ถูกเรียกใช้งานในทุก query เพื่อช่วยลดความซ้ำซ้อนของข้อมูลและสอดคล้องกับหลัก SRP
- **ความสัมพันธ์แบบ One-to-Many (1:N)** : เป็นความสัมพันธ์ที่ข้อมูล 1 รายการสามารถมีข้อมูลย่อยที่เกี่ยวข้องได้หลายรายการ เช่น Product (1) สามารถมีรายการรีวิวสินค้าได้หลายรายการ (Review) ซึ่งใช้สำหรับข้อมูลลูกที่สามารถเพิ่มจำนวนได้อย่างไม่จำกัดและออกแบบตามหลัก OCP

### 3. Strategy Pattern สำหรับคำนวณส่วนลด

ระบบได้นำ **Strategy Pattern** มาใช้ในการจัดการคำนวณส่วนลดสินค้า เพื่อแยกอัลกอริทึมการคำนวณออกจากคลาสหลัก โดยมี **DiscountStrategy** เป็น Interface กลาง และสร้างคลาสย่อยเพื่อรองรับพฤติกรรมที่แตกต่างกัน เช่น **NoDiscountStrategy**, **MemberDiscountStrategy** (ส่วนลด 10%) และ **SeasonalSaleStrategy** (ส่วนลด 20%) โดยควบคุมการทำงานผ่าน **DiscountContext** ในชั้น **Service**

### 4. Execution Flow (HTTP Request → DB)

ขั้นตอนการทำงานของระบบตั้งแต่ผู้ใช้งานส่งคำขอเข้ามาจนถึงฐานข้อมูล มีลำดับดังนี้

1. **HTTP Request** : ผู้ใช้งานส่งคำขอผ่านเว็บเบราว์เซอร์ไปยัง **ProductController** (เช่น การเรียกดูรายการสินค้าหรือบันทึกข้อมูล)
2. **Business Logic & Strategy** : Controller เรียกใช้งานเมธอดใน **ProductService** เพื่อจัดการตรรกะทางธุรกิจและทำการคำนวณราคาหรือส่วนลดผ่าน Strategy Pattern
3. **Repository Interface** : Service ส่งคำสั่งผ่าน **ProductRepository** (ซึ่งขยายมาจาก Spring Data JPA) เพื่อจัดการข้อมูล
4. **Database (PostgreSQL)** : Hibernate ทำหน้าที่แปลงคำสั่ง Java เป็น SQL และทำการบันทึกหรือดึงข้อมูลลงสู่ฐานข้อมูล **PostgreSQL (lab8demo)** ก่อนจะส่งผลลัพธ์กลับมาแสดงผลบนหน้าจอ

## ส่วนที่ 2: Code

### 1. Entity ทั้ง 3 ตัว :

- Product.java (Entity หลักที่เป็นเจ้าของความสัมพันธ์ทั้ง 1:1 และ 1:N)

```
package com.example.demo.model;

import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column private String name;
    @Column private String category;
    @Column private String brand;
    @Column private Integer stock;
    @Column private Double price;
    @Column private String discountType;

    // — 1:1 กับ ProductDetail —
    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "detail_id", referencedColumnName = "id")
    private ProductDetail detail;

    // — 1:N กับ Review —
    @OneToMany(mappedBy = "product", cascade = CascadeType.ALL)
    private List<Review> reviews = new ArrayList<>();
}
```

```
// Getters and Setters
public Long getId() { return id; }
public void setId(Long id) { this.id = id; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public String getCategory() { return category; }
public void setCategory(String category) { this.category = category; }
public String getBrand() { return brand; }
public void setBrand(String brand) { this.brand = brand; }
public Integer getStock() { return stock; }
public void setStock(Integer stock) { this.stock = stock; }
public Double getPrice() { return price; }
public void setPrice(Double price) { this.price = price; }
public String getDiscountType() { return discountType; }
public void setDiscountType(String discountType) { this.discountType = discountType; }
public ProductDetail getDetail() { return detail; }
public void setDetail(ProductDetail detail) { this.detail = detail; }
public List<Review> getReviews() { return reviews; }
public void setReviews(List<Review> reviews) { this.reviews = reviews; }
}
```

#### คำอธิบาย Annotation :

- @Entity : ระบุว่าคลาสนี้เป็น JPA Entity สำหรับแมปกับตารางในฐานข้อมูล
- @Table(name = "products") : กำหนดชื่อตารางในฐานข้อมูลเป็น products
- @Id และ @GeneratedValue(strategy = GenerationType.IDENTITY) : กำหนดให้ฟิลด์ id เป็น Primary Key และให้ฐานข้อมูลจัดการเพิ่มค่าอัตโนมัติ
- @OneToOne(cascade = CascadeType.ALL) ร่วมกับ @JoinColumn : กำหนดความสัมพันธ์แบบ 1:1 โดยให้ตาราง products เป็นเจ้าของ Foreign Key (detail\_id) และเมื่อมีการบันทึกหรือลบข้อมูล Product จะส่งผลกระทบต่อ (Cascade) ไปยัง ProductDetail ด้วย
- @OneToMany(mappedBy = "product", cascade = CascadeType.ALL) : กำหนดความสัมพันธ์แบบ 1:N ไปยังตาราง Review โดยระบุว่าถูกผูกด้วยฟิลด์ product ทางฝั่ง Review

- ProductDetail.java (ข้อมูลรายละเอียดสินค้าแบบ 1:1)

```
package com.example.demo.model;

import jakarta.persistence.*;

@Entity
@Table(name = "product_details")
public class ProductDetail {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column private String description;
    @Column private String warranty;
    @Column private Double weight;
    @Column private String dimensions;
    @Column private String manufacturedCountry;

    @OneToOne(mappedBy = "detail")
    private Product product;

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
    public String getWarranty() { return warranty; }
    public void setWarranty(String warranty) { this.warranty = warranty; }
    public Double getWeight() { return weight; }
    public void setWeight(Double weight) { this.weight = weight; }
    public String getDimensions() { return dimensions; }
    public void setDimensions(String dimensions) { this.dimensions = dimensions; }
    public String getManufacturedCountry() { return manufacturedCountry; }
    public void setManufacturedCountry(String manufacturedCountry) { this.manufacturedCountry =
manufacturedCountry; }

    public Product getProduct() { return product; }
    public void setProduct(Product product) { this.product = product; }
}
```

### คำอธิบาย Annotation:

- @OneToOne(mappedBy = "detail") : ระบุว่าเป็นฝั่ง Inverse (ไม่ใช่เจ้าของ Foreign Key โดยตรง) โดยมีความสัมพันธ์ย้อนกลับไปยังฟิลด์ detail ในคลาส Product
- Review.java (ข้อมูลรีวิวสินค้าแบบ 1:N)

```
package com.example.demo.model;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
@Table(name = "reviews")
public class Review {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column private String reviewer;
    @Column private Integer rating;
    @Column private String comment;
    @Column private LocalDate reviewDate;

    @ManyToOne
    @JoinColumn(name = "product_id")
    private Product product;
```

```
// Getters and Setters
public Long getId() { return id; }
public void setId(Long id) { this.id = id; }
public String getReviewer() { return reviewer; }
public void setReviewer(String reviewer) { this.reviewer = reviewer; }
public Integer getRating() { return rating; }
public void setRating(Integer rating) { this.rating = rating; }
public String getComment() { return comment; }
public void setComment(String comment) { this.comment = comment; }
public LocalDate getReviewDate() { return reviewDate; }
public void setReviewDate(LocalDate reviewDate) { this.reviewDate = reviewDate; }
public Product getProduct() { return product; }
public void setProduct(Product product) { this.product = product; }
}
```

**คำอธิบาย Annotation:**

- @ManyToOne และ @JoinColumn(name = "product\_id") : ฝั่ง Many จะเป็นผู้เก็บ Foreign Key (product\_id) เสมอ เพื่อเชื่อมโยงกลับไปยังสินค้าที่เป็นเจ้าของรีวิวนั้น ๆ

## 2. Service และ Controller พร้อมอธิบาย Constructor Injection :

- ProductService.java (ชั้น Business Logic)

```
package com.example.demo.service;

import com.example.demo.model.Product;
import com.example.demo.repository.ProductRepository;
import com.example.demo.strategy.*;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;

@Service
public class ProductService {

    private final ProductRepository productRepository;

    // Constructor Injection
    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public List<Product> getAllProducts() { return productRepository.findAll(); }
    public Optional<Product> getProductById(Long id) { return productRepository.findById(id); }
    public Product saveProduct(Product product) { return productRepository.save(product); }
    public void deleteProduct(Long id) { productRepository.deleteById(id); }

    public double calculateFinalPrice(Product product) {
        DiscountContext context = new DiscountContext();
        String type = product.getDiscountType();
        if ("MEMBER".equals(type)) {
            context.setStrategy(new MemberDiscountStrategy());
        } else if ("SEASONAL".equals(type)) {
            context.setStrategy(new SeasonalSaleStrategy());
        } else {
            context.setStrategy(new NoDiscountStrategy());
        }
        return context.executeDiscount(product.getPrice());
    }
}
```



- ProductController.java (ชั้นควบคุมเส้นทาง HTTP Request)

```
package com.example.demo.controller;

import com.example.demo.model.Product;
import com.example.demo.service.ProductService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping("/products")
public class ProductController {
    private final ProductService productService;

    // Constructor Injection
    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping
    public String listProducts(Model model) {
        model.addAttribute("products", productService.getAllProducts());
        return "products/list";
    }

    @GetMapping("/add")
    public String addProductForm(Model model) {
        model.addAttribute("product", new Product());
        return "products/add";
    }

    @PostMapping("/save")
    public String saveProduct(@ModelAttribute Product product) {
        productService.saveProduct(product);
        return "redirect:/products";
    }
}
```

```

@GetMapping("/edit/{id}")
public String editProductForm(@PathVariable Long id, Model model) {
    Product product = productService.getProductById(id)
        .orElseThrow(() -> new IllegalArgumentException("Invalid product Id:" + id));
    model.addAttribute("product", product);
    return "products/edit";
}

@PostMapping("/update/{id}")
public String updateProduct(@PathVariable Long id, @ModelAttribute Product product) {
    product.setId(id);
    productService.saveProduct(product);
    return "redirect:/products";
}

@GetMapping("/delete/{id}")
public String deleteProductForm(@PathVariable Long id, Model model) {
    Product product = productService.getProductById(id)
        .orElseThrow(() -> new IllegalArgumentException("Invalid product Id:" + id));
    model.addAttribute("product", product);
    return "products/delete";
}

@PostMapping("/delete/{id}")
public String deleteProduct(@PathVariable Long id) {
    productService.deleteProduct(id);
    return "redirect:/products";
}
}

```

### คำอธิบาย Constructor Injection :

- Constructor Injection คือรูปแบบการส่งผ่าน Dependency (เช่น ProductRepository ใน Service หรือ ProductService ใน Controller) ผ่านทาง Constructor ของคลาสโดยตรง โดยใช้คีย์เวิร์ด final เพื่อบังคับให้ตัวแปรไม่สามารถเปลี่ยนแปลงค่าได้หลังถูกสร้างขึ้น
- ข้อดี : ช่วยให้โค้ดมีความปลอดภัยสูงขึ้น (Immutable), ง่ายต่อการทำ Unit Testing (สามารถ Mock Dependency ได้ง่ายโดยไม่ต้องพึ่งพา Spring Container) และช่วยลดการผูกมัด (Loose Coupling) ระหว่างคลาสตามหลักการของ Dependency Inversion Principle (DIP)

### ส่วนที่ 3: ภาพหน้าจอการทำงาน (Screen Captures)

ในการบันทึกข้อมูลทดสอบระบบ ได้กำหนดชื่อสินค้าทดสอบในฟอร์มให้มีรหัสนักศึกษาและ Section ต่อท้ายอย่างชัดเจน เช่น iPhone 15 Pro (673380123-4 SEC 1)

- **Create** : กรอกข้อมูล Product + ProductDetail

**+ เพิ่มสินค้าใหม่**

**ข้อมูลสินค้า (Product)**

ชื่อสินค้า: iPhone 15 Pro หมวดหมู่: Electronics

แบรนด์: Apple จำนวนในสต็อก: 26

ราคาปกติ (บาท): 23990 ประเภทส่วนลด: ไม่มีส่วนลด

**ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1**

รายละเอียดสินค้า: มาพร้อมชิป Apple A17 Pro ดีไซน์ไทเทเนียมเกรนเดียวกับนาฬิกา หน้าจอ 6.1 นิ้ว และพอร์ตชาร์จ USB-C

ระยะเวลาประกัน: 1 Year น้ำหนัก (กิโลกรัม): 0.19

ขนาด (กว้าง x ยาว x สูง): 7.06 x 14.66 x 0.825 cm. ประเทศต้นกำเนิด: China

**รีวิวสินค้า (Review) — ความสัมพันธ์ 1:N**

เพื่อรีวิวผลิตภัณฑ์สินค้า (สามารถเพิ่มรีวิวสินค้าของคุณได้)

ชื่อผู้รีวิว: ชูโร สมานแก้ว คะแนน (1-5): ★★★★★ (4)

ความเห็น: ใช้ได้ 3 เดือนและพอใจเป็นใจและสะดวก

**บันทึกสินค้า** **ยกเลิก**

Product S

+ เพิ่มสินค้าใหม่

# ชื่อสินค้า

+ เพิ่มสินค้าใหม่

# ชื่อสินค้า

+ เพิ่มสินค้าใหม่

# ชื่อสินค้า

ชื่อสินค้า

หมวดหมู่

ยี่ห้อ

จำนวนในสต็อก

ราคาปกติ (บาท)

ประเภทส่วนลด

ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

ระยะเวลาประกัน

น้ำหนัก (กิโลกรัม)

ขนาด (กว้าง x ยาว x สูง)

ประเทศต้นกำเนิด

รีวิวสินค้า (Review) — ความสัมพันธ์ 1:N

เป็นผู้รีวิว

คะแนน (1-5)

ความเห็น

บันทึกสินค้า

ยกเลิก

- Read : รายการสินค้าในตาราง

Product Shop

Lab 8 — 1:1 & 1:N Relationship

+ เพิ่มสินค้าใหม่

| # | ชื่อสินค้า    | หมวดหมู่    | ยี่ห้อ | คลัง | ราคา (บาท) | ส่วนลด | ราคาสุทธิ | วันประกัน (ปี) | น้ำหนัก (กก) | รีวิว (1/5) | จัดการ                                |
|---|---------------|-------------|--------|------|------------|--------|-----------|----------------|--------------|-------------|---------------------------------------|
| 2 | iPhone 15 Pro | Electronics | Apple  | 26   | 23990.00   | NONE   | 23990.00  | 1 Year         | 0.19         | 1 รีวิว     | <div><div>ลบ</div><div>ลบ</div></div> |

- Update : หน้าฟอร์มแก้ไข

Product Shop

+ เพิ่มสินค้าใหม่

# ชื่อสินค้า

2 iPhone 15 Pro

ข้อมูลสินค้า (Product)

ชื่อสินค้า

iPhone 15 Pro

หมวดหมู่

Electronics

ยี่ห้อ

Apple

จำนวนสินค้า

26

ราคาปลีก (บาท)

23990.0

ประเภทส่วนลด

ไม่มีส่วนลด

ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

มาพร้อมชิป Apple A17 Pro ที่ใช้พลังงานน้อยกว่าขนาดหน้าจอกว้าง 6.1 นิ้ว และเปลี่ยนมาใช้พอร์ต USB-C

ระยะเวลาประกัน

1 Year

น้ำหนัก (กิโลกรัม)

0.19

ขนาด

7.06 x 14.66 x 0.825 cm.

ประเทศต้นกำเนิด

China

บันทึกการแก้ไข

ยกเลิก

Product Shop

+ เพิ่มสินค้าใหม่

# ชื่อสินค้า

2 iPhone 15 Pro

ข้อมูลสินค้า (Product)

ชื่อสินค้า

iPhone 15 Pro

หมวดหมู่

Electronics

ยี่ห้อ

Apple

จำนวนสินค้า

26

ราคาปลีก (บาท)

23990.0

ประเภทส่วนลด

ส่วนลดพิเศษ (20%)

ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

มาพร้อมชิป Apple A17 Pro ที่ใช้พลังงานน้อยกว่าขนาดหน้าจอกว้าง 6.1 นิ้ว และเปลี่ยนมาใช้พอร์ต USB-C

ระยะเวลาประกัน

1 Year

น้ำหนัก (กิโลกรัม)

0.19

ขนาด

7.06 x 14.66 x 0.825 cm.

ประเทศต้นกำเนิด

China

บันทึกการแก้ไข

ยกเลิก

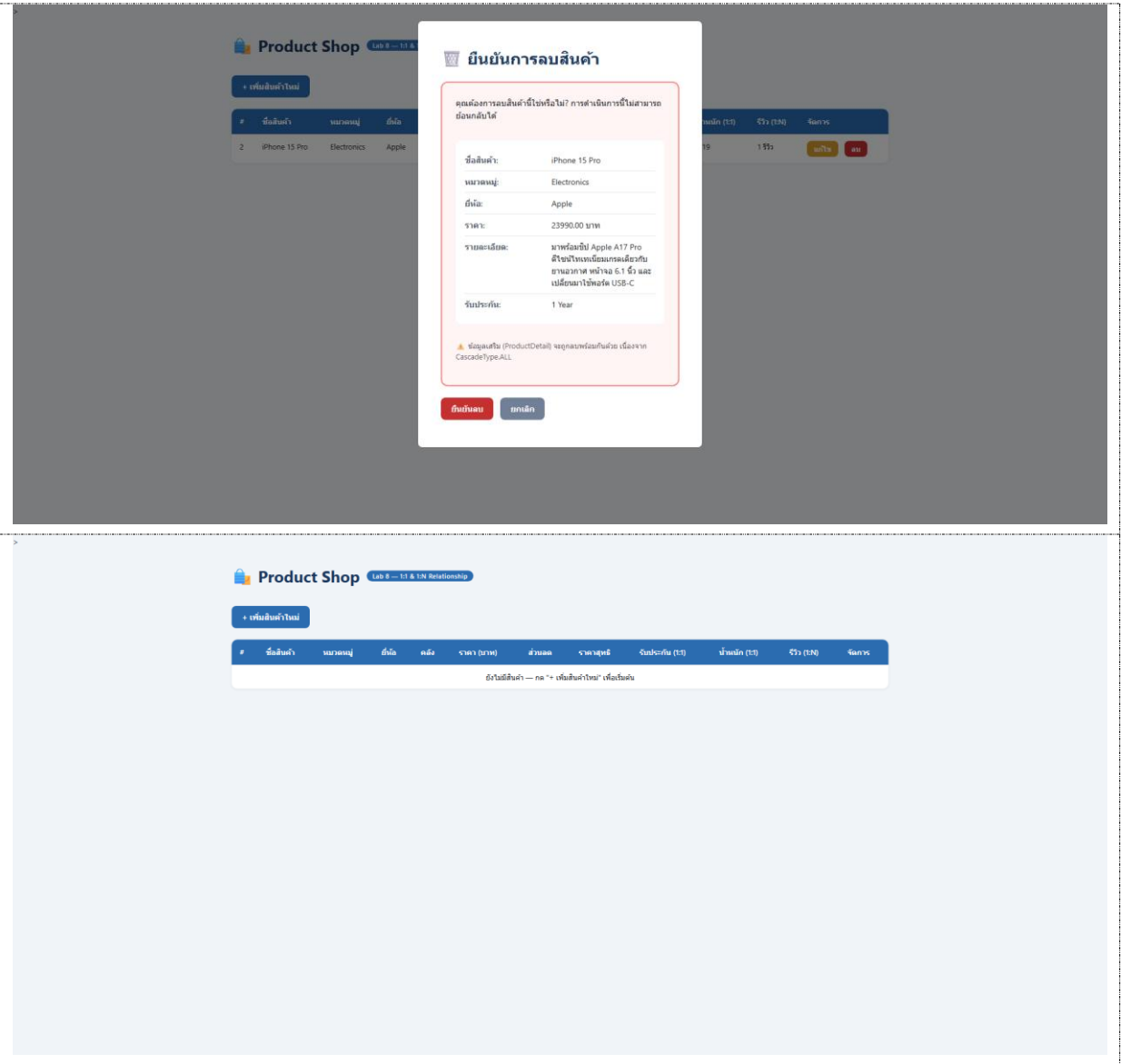
Product Shop

Lab 8 — 1:1 & O-N Relationship

+ เพิ่มสินค้าใหม่

| # | ชื่อสินค้า    | หมวดหมู่    | ยี่ห้อ | ตัว | ราคา (บาท) | ส่วนลด   | ราคาสุทธิ | ระยะเวลา (ปี) | น้ำหนัก (กิโลกรัม) | ตัว (kg) | จัดการ             |
|---|---------------|-------------|--------|-----|------------|----------|-----------|---------------|--------------------|----------|--------------------|
| 2 | iPhone 15 Pro | Electronics | Apple  | 26  | 23990.00   | SEASONAL | 19192.00  | 1 Year        | 0.19               | 1 ปี     | <div>แก้ไขลบ</div> |

- Delete : ยืนยันลบ + กลับ



- Database : pgAdmin แสดง 3 ตาราง (products, product\_details, reviews) พร้อม FK

